

Лабораторная работа (Вариант 1)

Python + PyQt6 + MySQL (VS Code)

Автор: Каламбет В.Б.

подготовка к демонстрационному экзамену

Содержание

Шаг 1. Создайте рабочую структуру проекта	4
Шаг 2. Поднимите MySQL + phpMyAdmin через Docker Compose	4
Шаг 3. Создайте схему БД (модули 1–4)	5
Шаг 4. Подготовьте CSV из исходных XLSX (Через Excel)	7
Шаг 5. Импортируйте данные через UI phpMyAdmin	8
Шаг 5.1. Перенос raw-данных в боевые таблицы	8
Шаг 6. Выполните контрольные SQL-проверки	9
Шаг 7. Создайте Python-проект и установите зависимости	10
Шаг 8. Создайте db.py	10
Шаг 8.1. Создайте check_db.py и проверьте подключение	10
Шаг 9. Создайте calculations.py	11
Шаг 10. Создайте main.py (все окна приложения)	13
Шаг 11. Запустите приложение	22
Шаг 12. Проверьте контрольные значения расчета стоимости партии	23
Шаг 13. Проверьте метод модуля 4	23
Шаг 14. Подготовьте локальный git-коммит для метода модуля 4	24
Шаг 15. Экспортируйте SQL-скрипт БД	24
Шаг 16. Сохраните ER-диаграмму БД в PDF	24
Шаг 17. Соберите исполняемый файл .exe	25
Шаг 18. Подготовьте финальный набор файлов для локального репозитория .	25

Результат лабораторной

После выполнения у вас будет готовое приложение по модулям 1–4:

- база данных MySQL в 3НФ;
 - ER-диаграмма БД в PDF;
 - импорт исходных данных из `xlsx` (через `csv`) в `phpMyAdmin`;
 - главная форма со списком материалов и расчетом стоимости минимальной партии;
 - форма добавления/редактирования материала;
 - окно поставщиков выбранного материала;
 - метод расчета количества продукции (модуль 4) и его локальный `git`-коммит;
 - SQL-скрипт БД, исполняемый файл приложения и набор итоговых файлов для локального репозитория.
-

Шаг 1. Создайте рабочую структуру проекта

Что делаем

Создайте папку проекта и подкаталоги для кода и ресурсов.

Команды/код

```
cd C:\
mkdir demoexam_python
cd demoexam_python
mkdir resources
```

Скопируйте из архива в папку resources:

- Мозаика.png
 - Мозаика.ico
 - все файлы *_import.xlsx
-

Шаг 2. Поднимите MySQL + phpMyAdmin через Docker Compose

Что делаем

Создайте docker-compose.yml и запустите контейнеры. Альтернативно (без Docker) можно использовать XAMPP, Open Server Panel или локально установленный MySQL Server.

Команды/код

Создайте файл C:\demoexam_python\docker-compose.yml:

```
services:
  mysql:
    image: mysql:8.4
    container_name: demoexam_mysql
    restart: unless-stopped
    environment:
      MYSQL_ROOT_PASSWORD: root
      MYSQL_DATABASE: mosaic_demo
      MYSQL_USER: demo
      MYSQL_PASSWORD: demo
    ports:
      - "3306:3306"
    volumes:
      - mysql_data:/var/lib/mysql

  phpmyadmin:
    image: phpmyadmin:latest
    container_name: demoexam_phpmyadmin
    restart: unless-stopped
```

```
environment:
  PMA_HOST: mysql
  PMA_PORT: 3306
  PMA_USER: root
  PMA_PASSWORD: root
ports:
  - "8081:80"
depends_on:
  - mysql

volumes:
  mysql_data:
```

Запуск:

```
cd C:\demoexam_python
docker compose up -d
docker compose ps
```

Шаг 3. Создайте схему БД (модули 1–4)

Что делаем

В phpMyAdmin создайте таблицы, связи, ограничения и служебные raw-таблицы для импорта.

Команды/код

1. Откройте phpMyAdmin:
 - при Docker: `http://localhost:8081`;
 - при XAMPP / Open Server Panel / локальном MySQL Server: адрес phpMyAdmin вашей локальной установки (часто `http://localhost/phpmyadmin`).
2. Войдите под пользователем с правами на создание таблиц (для Docker: `root / root`).
3. Если БД `mosaic_demo` уже есть — выберите ее.
4. Если БД `mosaic_demo` нет: вкладка **Базы данных** -> введите имя `mosaic_demo` -> выберите сравнение `utf8mb4_unicode_ci` -> нажмите **Создать** -> откройте созданную БД.
5. Откройте вкладку **SQL** и выполните скрипт:

```
USE mosaic_demo;
```

```
SET NAMES utf8mb4;
```

```
DROP TABLE IF EXISTS material_suppliers;
DROP TABLE IF EXISTS materials;
DROP TABLE IF EXISTS suppliers;
DROP TABLE IF EXISTS product_types;
DROP TABLE IF EXISTS material_types;
```

```

DROP TABLE IF EXISTS materials_import_raw;
DROP TABLE IF EXISTS material_suppliers_import_raw;

CREATE TABLE material_types (
    material_type_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL UNIQUE,
    loss_percent DECIMAL(10,4) NOT NULL CHECK (loss_percent >= 0)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE product_types (
    product_type_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL UNIQUE,
    coefficient DECIMAL(10,2) NOT NULL CHECK (coefficient > 0)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE suppliers (
    supplier_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(150) NOT NULL UNIQUE,
    supplier_type VARCHAR(50) NOT NULL,
    inn VARCHAR(12) NOT NULL UNIQUE,
    rating INT NOT NULL CHECK (rating >= 0),
    start_date DATE NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE materials (
    material_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(150) NOT NULL UNIQUE,
    material_type_id INT NOT NULL,
    unit_price DECIMAL(12,2) NOT NULL CHECK (unit_price >= 0),
    stock_quantity INT NOT NULL CHECK (stock_quantity >= 0),
    min_quantity INT NOT NULL CHECK (min_quantity >= 0),
    package_quantity INT NOT NULL CHECK (package_quantity > 0),
    unit_name VARCHAR(20) NOT NULL,
    CONSTRAINT fk_materials_type
        FOREIGN KEY (material_type_id) REFERENCES
material_types(material_type_id)
        ON UPDATE CASCADE ON DELETE RESTRICT
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE material_suppliers (
    material_id INT NOT NULL,
    supplier_id INT NOT NULL,
    PRIMARY KEY (material_id, supplier_id),
    CONSTRAINT fk_ms_material
        FOREIGN KEY (material_id) REFERENCES materials(material_id)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    CONSTRAINT fk_ms_supplier
        FOREIGN KEY (supplier_id) REFERENCES suppliers(supplier_id)
        ON UPDATE CASCADE ON DELETE RESTRICT

```

```

) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE INDEX idx_materials_type ON materials(material_type_id);
CREATE INDEX idx_ms_supplier ON material_suppliers(supplier_id);

-- raw-таблицы только для удобного импорта csv с текстовыми названиями
CREATE TABLE materials_import_raw (
    name VARCHAR(150) NOT NULL,
    material_type_name VARCHAR(100) NOT NULL,
    unit_price DECIMAL(12,2) NOT NULL,
    stock_quantity INT NOT NULL,
    min_quantity INT NOT NULL,
    package_quantity INT NOT NULL,
    unit_name VARCHAR(20) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE material_suppliers_import_raw (
    material_name VARCHAR(150) NOT NULL,
    supplier_name VARCHAR(150) NOT NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

```

Шаг 4 Подготовьте CSV из исходных XLSX (Через Excel)

Что делаем

Откройте каждый xlsx в Excel, сохраните как CSV UTF-8 и удалите строку заголовков.

Команды/код

Сохраните файлы:

- Material_type_import.csv
- Product_type_import.csv
- Suppliers_import.csv
- Materials_import.csv
- Material_suppliers_import.csv

В каждом CSV удалите первую строку (заголовки), чтобы файл начинался сразу с данных.

Важно:

1. Material_type_import.csv:
 - Во втором столбце должны быть значения вида 0.0012 (без знака %).
 - Если Excel показывает 0,12%, смените формат столбца на Общий или Числовой, затем замените запятую , на точку . (пример: 0,12 → 0.12).
2. Product_type_import.csv:
 - Во втором столбце замените , на . (пример: 8.59).
3. Suppliers_import.csv:

- Дата должна быть в формате YYYY-MM-DD (пример: 2015-12-20).
 - Проверьте, что ИНН не в экспоненциальной записи (9.432E+09), а обычным числом.
4. Materials_import.csv:
- Замените , на . в дробных значениях.
 - Выделите только целые колонки (Количество на складе, Минимальное количество, Количество в упаковке) и нажмите Уменьшить разрядность несколько раз, чтобы убрать хвосты ,00.
5. Material_suppliers_import.csv:
- Дополнительных числовых правок не требуется.

Шаг 5. Импортируйте данные через UI phpMyAdmin

Что делаем

Импортируйте CSV в таблицы в правильном порядке.

Команды/код

В phpMyAdmin:

1. Откройте таблицу material_types -> **Импорт** -> файл Material_type_import.csv -> поле Названия столбцов: name,loss_percent.
2. Откройте таблицу product_types -> **Импорт** -> файл Product_type_import.csv -> поле Названия столбцов: name,coefficient.
3. Откройте таблицу suppliers -> **Импорт** -> файл Suppliers_import.csv -> поле Названия столбцов: name,supplier_type,inn,rating,start_date.
4. Откройте таблицу materials_import_raw -> **Импорт** -> файл Materials_import.csv -> поле Названия столбцов: name,material_type_name,unit_price,stock_quantity,min_quantity,package_
5. Откройте таблицу material_suppliers_import_raw -> **Импорт** -> файл Material_suppliers_import.csv -> поле Названия столбцов: material_name,supplier_name.

Для каждого импорта обязательно выставьте:

- Формат = CSV.
- Разделитель полей = ; (для этой ЛР). Если ваш CSV с запятыми, поставьте ,.
- Значения полей обрамлены = ".
- Символ экранирования = " (оставьте значение по умолчанию в вашей версии).
- Разделитель строк = auto.
- Названия столбцов — заполните вручную, как указано в пунктах 1–5 выше.

Шаг 5.1. Перенос raw-данных в боевые таблицы

Что делаем

После импорта raw-таблиц перенесите данные в рабочие таблицы со связями.

Команды/код

Во вкладке **SQL** выполните:

```
USE mosaic_demo;
```

```
INSERT INTO materials (
    name, material_type_id, unit_price, stock_quantity, min_quantity,
    package_quantity, unit_name
)
SELECT
    r.name,
    mt.material_type_id,
    r.unit_price,
    r.stock_quantity,
    r.min_quantity,
    r.package_quantity,
    r.unit_name
FROM materials_import_raw r
JOIN material_types mt ON mt.name = r.material_type_name;

INSERT INTO material_suppliers (material_id, supplier_id)
SELECT
    m.material_id,
    s.supplier_id
FROM material_suppliers_import_raw r
JOIN materials m ON m.name = r.material_name
JOIN suppliers s ON s.name = r.supplier_name;
```

Шаг 6. Выполните контрольные SQL-проверки

Что делаем

Проверьте, что импорт соответствует целевым количествам.

Команды/код

```
USE mosaic_demo;
```

```
SELECT 'material_types' AS table_name, COUNT(*) AS cnt FROM material_types
UNION ALL
SELECT 'product_types', COUNT(*) FROM product_types
UNION ALL
SELECT 'suppliers', COUNT(*) FROM suppliers
UNION ALL
SELECT 'materials', COUNT(*) FROM materials
UNION ALL
SELECT 'material_suppliers', COUNT(*) FROM material_suppliers;
```

Шаг 7. Создайте Python-проект и установите зависимости

Что делаем

Создайте виртуальное окружение и установите пакеты.

Команды/код

```
cd C:\demoexam_python
python -m venv .venv
.\.venv\Scripts\activate
pip install PyQt6 mysql-connector-python pyinstaller
pip freeze > requirements.txt
```

Шаг 8. Создайте db.py

Что делаем

Вынесите подключение к БД в отдельный модуль.

Команды/код

Создайте C:\demoexam_python\db.py:

```
import mysql.connector

DB_CONFIG = {
    "host": "127.0.0.1",
    "port": 3306,
    "user": "demo",
    "password": "demo",
    "database": "mosaic_demo",
    "use_unicode": True,
    "charset": "utf8mb4",
    "use_pure": True,
}

def get_connection():
    return mysql.connector.connect(**DB_CONFIG)
```

Важно

db.py отдельно не запускаем: это модуль подключения.

Проверка выполняется в следующем шаге через check_db.py. Если используете ХАМРР: обычно user = "root", password = "".

Шаг 8.1. Создайте check_db.py и проверьте подключение

Что делаем

Создайте отдельный файл для проверки подключения к БД.

Команды/код

Создайте C:\demoexam_python\check_db.py:

```
from db import get_connection

def main():
    conn = None
    cur = None
    try:
        conn = get_connection()
        cur = conn.cursor()
        cur.execute("SELECT 1")
        result = cur.fetchone()
        print(f"OK: подключение успешно, SELECT 1 -> {result[0]}")
    except Exception as e:
        print("ERROR:", e)
    finally:
        if cur is not None:
            cur.close()
        if conn is not None and conn.is_connected():
            conn.close()

if __name__ == "__main__":
    main()
```

Запуск:

```
cd C:\demoexam_python
python check_db.py
```

Шаг 9. Создайте calculations.py

Что делаем

Реализуйте 2 расчетных метода:

- стоимость минимально необходимой партии;
- количество продукции (модуль 4).
- Отдельно calculations.py не запускаем: это модуль с функциями.

Команды/код

Создайте C:\demoexam_python\calculations.py:

```
from decimal import Decimal, ROUND_HALF_UP
from math import ceil

from db import get_connection

def calculate_min_purchase_cost(
    stock_quantity: int,
```

```

        min_quantity: int,
        package_quantity: int,
        unit_price: Decimal,
    ) -> Decimal:
        if stock_quantity >= min_quantity:
            return Decimal("0.00")

        deficit = min_quantity - stock_quantity
        purchase_qty = ceil(deficit / package_quantity) * package_quantity
        cost = Decimal(purchase_qty) * Decimal(unit_price)
        if cost < 0:
            return Decimal("0.00")
        return cost.quantize(Decimal("0.01"), rounding=ROUND_HALF_UP)

def calculate_products_count(
    product_type_id: int,
    material_type_id: int,
    raw_amount: int,
    param1: float,
    param2: float,
) -> int:
    if (
        product_type_id <= 0
        or material_type_id <= 0
        or raw_amount < 0
        or param1 <= 0
        or param2 <= 0
    ):
        return -1

    conn = get_connection()
    try:
        cur = conn.cursor(dictionary=True)
        cur.execute(
            """
            SELECT
                pt.coefficient,
                mt.loss_percent
            FROM product_types pt
            JOIN material_types mt
            ON mt.material_type_id = %s
            WHERE pt.product_type_id = %s
            """,
            (material_type_id, product_type_id),
        )
        row = cur.fetchone()
        if row is None:
            return -1

```

```

        raw_per_unit = Decimal(str(param1)) * Decimal(str(param2)) *
Decimal(str(row["coefficient"]))
        raw_per_unit = raw_per_unit * (Decimal("1") +
Decimal(str(row["loss_percent"])))
        if raw_per_unit <= 0:
            return -1

        return int(Decimal(raw_amount) // raw_per_unit)
    finally:
        conn.close()

```

Шаг 10. Создайте main.py (все окна приложения)

Что делаем

Добавьте UI:

- главная форма со списком материалов;
- форма добавления/редактирования;
- окно поставщиков материала.

Команды/код

Создайте C:\demoexam_python\main.py:

```

import sys
from decimal import Decimal
from pathlib import Path

from PyQt6.QtCore import Qt
from PyQt6.QtGui import QFont, QIcon, QPixmap
from PyQt6.QtWidgets import (
    QApplication,
    QComboBox,
    QDialog,
    QFormLayout,
    QFrame,
    QGridLayout,
    QHBoxLayout,
    QLabel,
    QLineEdit,
    QMainWindow,
    QMessageBox,
    QPushButton,
    QScrollArea,
    QTableWidgetItem,
    QTableWidgetItemItem,
    QVBoxLayout,
    QWidget,
)

```

```

from calculations import calculate_min_purchase_cost
from db import get_connection

COLOR_MAIN_BG = "#FFFFFF"
COLOR_SECOND_BG = "#ABCFCE"
COLOR_ACCENT = "#546F94"

RESOURCES_DIR = Path(__file__).resolve().parent / "resources"
LOGO_PATH = RESOURCES_DIR / "Мозаика.png"
ICON_PATH = RESOURCES_DIR / "Мозаика.ico"

class MaterialCard(QFrame):
    def __init__(self, material: dict, on_edit, on_suppliers):
        super().__init__()
        self.setStyleSheet(
            f"""
            QFrame {{
                background-color: {COLOR_SECOND_BG};
                border: 1px solid #888888;
                border-radius: 6px;
            }}
            QLabel {{
                border: none;
                background: transparent;
            }}
            QPushButton {{
                background-color: {COLOR_ACCENT};
                color: white;
                border: none;
                padding: 6px 10px;
                border-radius: 4px;
            }}
            """
        )

        cost = calculate_min_purchase_cost(
            stock_quantity=material["stock_quantity"],
            min_quantity=material["min_quantity"],
            package_quantity=material["package_quantity"],
            unit_price=Decimal(str(material["unit_price"])),
        )

        root = QHBoxLayout(self)
        left = QVBoxLayout()
        right = QVBoxLayout()

        title = QLabel(f'{material["material_type"]} | {material["name"]}')
```

```

        left.addWidget(title)
        left.addWidget(QLabel(f'Минимальное количество:
{material["min_quantity"]} {material["unit_name"]}') )
        left.addWidget(QLabel(f'Количество на складе:
{material["stock_quantity"]} {material["unit_name"]}') )
        left.addWidget(
            QLabel(
                f'Цена:
{Decimal(str(material["unit_price"])).quantize(Decimal("0.01"))} p '
                f'/ Единица измерения: {material["unit_name"]}'
            )
        )

cost_label = QLabel(f"Стоимость партии: {cost} p")
cost_label.setStyleSheet("font-size: 24px; font-weight: 600;")
right.addWidget(cost_label, alignment=Qt.AlignmentFlag.AlignRight)

buttons = QHBoxLayout()
btn_edit = QPushButton("Редактировать")
btn_suppliers = QPushButton("Поставщики")
btn_edit.clicked.connect(lambda: on_edit(material["material_id"]))
btn_suppliers.clicked.connect(lambda:
on_suppliers(material["material_id"], material["name"]))
buttons.addWidget(btn_edit)
buttons.addWidget(btn_suppliers)
right.addLayout(buttons)
right.addStretch()

root.addLayout(left, stretch=4)
root.addLayout(right, stretch=2)

class MaterialFormDialog(QDialog):
    def __init__(self, material_types, material=None):
        super().__init__()
        self.material = material
        self.saved_data = None

        self.setWindowTitle("Добавление/редактирование материала")
        self.setMinimumWidth(520)

        layout = QVBoxLayout(self)
        form = QFormLayout()

        self.name_edit = QLineEdit()
        self.type_combo = QComboBox()
        for mt in material_types:
            self.type_combo.addItem(mt["name"], mt["material_type_id"])
        self.stock_edit = QLineEdit()
        self.unit_edit = QLineEdit()

```

```

self.package_edit = QLineEdit()
self.min_edit = QLineEdit()
self.price_edit = QLineEdit()

form.addRow("Наименование:", self.name_edit)
form.addRow("Тип материала:", self.type_combo)
form.addRow("Количество на складе:", self.stock_edit)
form.addRow("Единица измерения:", self.unit_edit)
form.addRow("Количество в упаковке:", self.package_edit)
form.addRow("Минимальное количество:", self.min_edit)
form.addRow("Цена единицы материала:", self.price_edit)
layout.addLayout(form)

buttons = QHBoxLayout()
save_btn = QPushButton("Сохранить")
cancel_btn = QPushButton("Отмена")
save_btn.clicked.connect(self.on_save)
cancel_btn.clicked.connect(self.reject)
buttons.addWidget(save_btn)
buttons.addWidget(cancel_btn)
layout.addLayout(buttons)

if material:
    self.name_edit.setText(material["name"])
    index = self.type_combo.findData(material["material_type_id"])
    if index >= 0:
        self.type_combo.setCurrentIndex(index)
    self.stock_edit.setText(str(material["stock_quantity"]))
    self.unit_edit.setText(material["unit_name"])
    self.package_edit.setText(str(material["package_quantity"]))
    self.min_edit.setText(str(material["min_quantity"]))
    self.price_edit.setText(str(material["unit_price"]))

def on_save(self):
    name = self.name_edit.text().strip()
    unit_name = self.unit_edit.text().strip()
    if not name or not unit_name:
        QMessageBox.warning(self, "Предупреждение", "Наименование и единица
измерения обязательны.")
        return

    try:
        stock_quantity = int(self.stock_edit.text().strip())
        package_quantity = int(self.package_edit.text().strip())
        min_quantity = int(self.min_edit.text().strip())
        unit_price = Decimal(self.price_edit.text().strip())
    except Exception:
        QMessageBox.critical(
            self,

```



```

        "Ошибка ввода",
        "Проверьте числовые поля: количество и цена должны быть
числами.",
    )
    return

    if stock_quantity < 0 or package_quantity <= 0 or min_quantity < 0 or
unit_price < 0:
        QMessageBox.critical(
            self,
            "Ошибка ввода",
            "Количество не может быть отрицательным, упаковка должна быть >
0, цена не может быть отрицательной.",
        )
        return

    if unit_price.as_tuple().exponent < -2:
        QMessageBox.warning(self, "Предупреждение", "Цена должна содержать не
более 2 знаков после запятой.")
        return

    self.saved_data = {
        "name": name,
        "material_type_id": self.type_combo.currentData(),
        "stock_quantity": stock_quantity,
        "unit_name": unit_name,
        "package_quantity": package_quantity,
        "min_quantity": min_quantity,
        "unit_price": unit_price,
    }
    self.accept()

class SuppliersDialog(QDialog):
    def __init__(self, material_name, rows):
        super().__init__()
        self.setWindowTitle(f"Поставщики материала: {material_name}")
        self.setMinimumWidth(700)

        layout = QVBoxLayout(self)
        table = QTableWidgetItem()
        table.setColumnCount(3)
        table.setHorizontalHeaderLabels(
            ["Наименование поставщика", "Рейтинг", "Дата начала работы"]
        )
        table.setRowCount(len(rows))

        for i, row in enumerate(rows):
            table.setItem(i, 0, QTableWidgetItem(str(row["name"])))
            table.setItem(i, 1, QTableWidgetItem(str(row["rating"])))

```

```

        table.setItem(i, 2, QTableWidgetItem(str(row["start_date"])))

    table.resizeColumnsToContents()
    layout.addWidget(table)

    back_btn = QPushButton("Назад")
    back_btn.clicked.connect(self.close)
    layout.addWidget(back_btn, alignment=Qt.AlignmentFlag.AlignRight)

class MainWindow(QMainWindow):
    def __init__(self):
        super().__init__()
        self.setWindowTitle("Учет материалов")
        self.resize(1200, 800)
        if ICON_PATH.exists():
            self.setWindowIcon(QIcon(str(ICON_PATH)))

        self.setStyleSheet(
            f"""
            QWidget {{
                font-family: 'Comic Sans MS';
                font-size: 13px;
                background-color: {COLOR_MAIN_BG};
            }}
            QPushButton {{
                background-color: {COLOR_ACCENT};
                color: white;
                border: none;
                padding: 8px 12px;
                border-radius: 4px;
            }}
            """
        )

        root = QWidget()
        root_layout = QVBoxLayout(root)

        header = QGridLayout()
        logo_label = QLabel()
        if LOGO_PATH.exists():
            logo_pix = QPixmap(str(LOGO_PATH)).scaled(96, 96,
Qt.AspectRatioMode.KeepAspectRatio)
            logo_label.setPixmap(logo_pix)
        title = QLabel("Список материалов")
        title.setStyleSheet("font-size: 26px; font-weight: 700;")
        add_btn = QPushButton("Добавить материал")
        add_btn.clicked.connect(self.add_material)
        refresh_btn = QPushButton("Обновить")
        refresh_btn.clicked.connect(self.load_materials)

```

```

header.addWidget(logo_label, 0, 0)
header.addWidget(title, 0, 1)
header.addWidget(add_btn, 0, 2)
header.addWidget(refresh_btn, 0, 3)
header.setColumnStretch(1, 1)
root_layout.addLayout(header)

self.cards_layout = QVBoxLayout()
self.cards_layout.setSpacing(14)
cards_container = QWidget()
cards_container.setLayout(self.cards_layout)

scroll = QScrollArea()
scroll.setWidgetResizable(True)
scroll.setWidget(cards_container)
root_layout.addWidget(scroll)

self.setCentralWidget(root)
self.load_materials()

def fetch_material_types(self):
    conn = get_connection()
    try:
        cur = conn.cursor(dictionary=True)
        cur.execute("SELECT material_type_id, name FROM material_types ORDER
BY name")
        return cur.fetchall()
    finally:
        conn.close()

def fetch_materials(self):
    conn = get_connection()
    try:
        cur = conn.cursor(dictionary=True)
        cur.execute(
            """
            SELECT
                m.material_id,
                m.name,
                m.material_type_id,
                mt.name AS material_type,
                m.unit_price,
                m.stock_quantity,
                m.min_quantity,
                m.package_quantity,
                m.unit_name
            FROM materials m
            JOIN material_types mt ON mt.material_type_id =

```

```

m.material_type_id
        ORDER BY m.material_id
        """
    )
    return cur.fetchall()
finally:
    conn.close()

def fetch_suppliers(self, material_id):
    conn = get_connection()
    try:
        cur = conn.cursor(dictionary=True)
        cur.execute(
            """
            SELECT s.name, s.rating, s.start_date
            FROM material_suppliers ms
            JOIN suppliers s ON s.supplier_id = ms.supplier_id
            WHERE ms.material_id = %s
            ORDER BY s.name
            """,
            (material_id,),
        )
        return cur.fetchall()
    finally:
        conn.close()

def clear_cards(self):
    while self.cards_layout.count():
        item = self.cards_layout.takeAt(0)
        widget = item.widget()
        if widget is not None:
            widget.deleteLater()

def load_materials(self):
    self.clear_cards()
    rows = self.fetch_materials()
    for row in rows:
        card = MaterialCard(row, self.edit_material, self.show_suppliers)
        self.cards_layout.addWidget(card)
    self.cards_layout.addStretch()

def add_material(self):
    material_types = self.fetch_material_types()
    dialog = MaterialFormDialog(material_types=material_types, material=None)
    if dialog.exec() == QDialog.DialogCode.Accepted:
        data = dialog.saved_data
        conn = get_connection()
        try:
            cur = conn.cursor()

```

```

cur.execute(
    """
        INSERT INTO materials
        (name, material_type_id, unit_price, stock_quantity,
min_quantity, package_quantity, unit_name)
        VALUES (%s, %s, %s, %s, %s, %s, %s)
    """,
    (
        data["name"],
        data["material_type_id"],
        str(data["unit_price"]),
        data["stock_quantity"],
        data["min_quantity"],
        data["package_quantity"],
        data["unit_name"],
    ),
)
conn.commit()
except Exception as ex:
    QMessageBox.critical(self, "Ошибка", f"Не удалось добавить
материал: {ex}")
finally:
    conn.close()
    self.load_materials()

def edit_material(self, material_id):
    rows = self.fetch_materials()
    material = next((x for x in rows if x["material_id"] == material_id),
None)
    if material is None:
        QMessageBox.critical(self, "Ошибка", "Материал не найден.")
        return

    material_types = self.fetch_material_types()
    dialog = MaterialFormDialog(material_types=material_types,
material=material)
    if dialog.exec() == QDialog.DialogCode.Accepted:
        data = dialog.saved_data
        conn = get_connection()
        try:
            cur = conn.cursor()
            cur.execute(
                """
                    UPDATE materials
                    SET
                        name = %s,
                        material_type_id = %s,
                        unit_price = %s,
                        stock_quantity = %s,

```

```

        min_quantity = %s,
        package_quantity = %s,
        unit_name = %s
WHERE material_id = %s
"""
    (
        data["name"],
        data["material_type_id"],
        str(data["unit_price"]),
        data["stock_quantity"],
        data["min_quantity"],
        data["package_quantity"],
        data["unit_name"],
        material_id,
    ),
)
conn.commit()
except Exception as ex:
    QMessageBox.critical(self, "Ошибка", f"Не удалось изменить
материал: {ex}")
finally:
    conn.close()
    self.load_materials()

def show_suppliers(self, material_id, material_name):
    rows = self.fetch_suppliers(material_id)
    dialog = SuppliersDialog(material_name, rows)
    dialog.exec()

def main():
    app = QApplication(sys.argv)
    app.setFont(QFont("Comic Sans MS", 11))
    win = MainWindow()
    win.show()
    sys.exit(app.exec())

if __name__ == "__main__":
    main()

```

Шаг 11. Запустите приложение

Что делаем

Проверьте, что UI запускается и загружает данные из БД.

Команды/код

```

cd C:\demoexam_python
.\.venv\Scripts\activate
python main.py

```

Шаг 12. Проверьте контрольные значения расчета стоимости партии

Что делаем

Отдельно выполните контрольную самопроверку значений перед сдачей.

Команды/код

Создайте C:\demoexam_python\check_costs.py:

```
from decimal import Decimal
from calculations import calculate_min_purchase_cost

print("Глина:", calculate_min_purchase_cost(1570, 5500, 30, Decimal("15.29")))
# ожидаем 60089.70
print("Монтмориллонит:", calculate_min_purchase_cost(3000, 3000, 30,
Decimal("17.6666667"))) # ожидаем 0.00
print("Жидкое стекло:", calculate_min_purchase_cost(500, 1500, 15,
Decimal("76.59")))      # ожидаем 76590.00
```

Запуск:

```
python check_costs.py
```

Шаг 13. Проверьте метод модуля 4

Что делаем

Проверьте calculate_products_count(...) на валидных и невалидных данных.

Команды/код

Создайте C:\demoexam_python\check_module4.py:

```
from calculations import calculate_products_count

print(calculate_products_count(1, 1, 1000, 2.5, 1.5)) # ожидаем 221
print(calculate_products_count(2, 4, 5000, 1.2, 2.1)) # ожидаем 230
print(calculate_products_count(4, 5, 12000, 3.0, 0.8)) # ожидаем 890
print(calculate_products_count(99, 1, 100, 1.0, 1.0)) # ожидаем -1
print(calculate_products_count(1, 1, 100, -1.0, 2.0)) # ожидаем -1
```

Запуск:

```
python check_module4.py
```

Шаг 14. Подготовьте локальный git-коммит для метода модуля 4

Что делаем

Зафиксируйте локальный коммит проекта после реализации метода модуля 4.

Команды/код

```
cd C:\demoexam_python
git init
git add .
git commit -m "Добавлен метод calculate_products_count (модуль 4)"
```

Шаг 15. Экпортируйте SQL-скрипт БД

Что делаем

Сохраните итоговый скрипт БД из phpMyAdmin.

Команды/код

1. В phpMyAdmin выберите БД mosaic_demo.
 2. Откройте вкладку **Экспорт**.
 3. В блоке **Метод экспорта** выберите Обычный - отображать все возможные настройки.
 4. Проверьте:
 - Формат = SQL;
 - в блоке Таблицы включены Структура и Данные;
 - в блоке Параметры создания объектов при необходимости включите Добавить выражение DROP TABLE / VIEW / PROCEDURE / FUNCTION / EVENT / TRIGGER и IF NOT EXISTS;
 - в блоке Параметры создания данных оператор = INSERT.
 5. В блоке Вывод оставьте Сохранить вывод в файл.
 6. Нажмите кнопку **Экспорт** и сохраните файл как mosaic_demo.sql.
-

Шаг 16. Сохраните ER-диаграмму БД в PDF

Что делаем

Получите ER-диаграмму средствами phpMyAdmin и сохраните в PDF (требование модуля 1).

Команды/код

1. В phpMyAdmin откройте БД mosaic_demo.
2. В верхнем меню БД выберите **Ещё** -> **Дизайнер**.
3. В дизайнере нажмите **Экспорт схемы**.
4. В поле формата выберите PDF.
5. Сохраните файл как mosaic_demo_er.pdf.

Шаг 17. Соберите исполняемый файл .exe

Что делаем

Сформируйте исполняемый файл приложения Python для сдачи.

Команды/код

```
cd C:\demoexam_python
pyinstaller --noconfirm --windowed --onefile --name MosaicMaterialsApp --icon
resources\Мозаика.ico --add-data "resources;resources" --collect-all
mysql.connector main.py
```

Важно

Перед повторной сборкой закройте запущенный MosaicMaterialsApp.exe, иначе возможна ошибка WinError 5 (Отказано в доступе) при перезаписи файла в dist.

Шаг 18. Подготовьте финальный набор файлов для локального репозитория

Что делаем

Соберите итоговые файлы точно в формате задания: исходники, исполняемый файл, SQL-скрипт, ER-PDF.

Команды/код

Подготовьте в папке проекта C:\demoexam_python:

- папка исходного кода C:\demoexam_python (структура файлов, не архив);
- C:\demoexam_python\dist\MosaicMaterialsApp.exe;
- mosaic_demo.sql (скопируйте экспортированный файл в C:\demoexam_python);
- mosaic_demo_er.pdf (скопируйте экспортированный файл в C:\demoexam_python).

Зафиксируйте итог в локальном git-репозитории:

```
cd C:\demoexam_python
git add .
git status
git commit -m "Финальная версия проекта (Python)"
```
